8

Управление объектами схемы

ORACLE

© Oracle, 2007. Все права защищены.

Задачи

После прохождения этого урока вы научитесь:

- определять объекты схемы и типы данных;
- создавать и изменять таблицы;
- определять ограничения;
- просматривать столбцы и содержимое таблиц;
- создавать индексы;
- создавать представления;
- создавать последовательности;
- объяснить назначение временных таблиц.

ORACLE

© Oracle, 2007. Все права защищены.

Что такое схема?

Схема – это набор объектов БД, принадлежащих определенному пользователю. При использовании баз данных в производстве пользователем обычно является не человек, а приложение. Схеме присваивается имя пользователя, который является ее владельцем. Объекты схемы – это логические структуры, которые напрямую ссылаются на данные БД. Объектами схемы являются такие структуры, как таблицы, представления и индексы.

Создавать и управлять объектами схемы можно с помощью языка SQL или утилиты Enterprise Manager. Если используется Enterprise Manager, то необходимый код SQL создается автоматически.

Примечание. Схема не обязательно непосредственно ссылается на одно табличное пространство. Можно определить конфигурации, в которых объекты одной схемы могут находиться в разных табличных пространствах, а табличное пространство может хранить объекты разных схем.

При создании базы данных также создается несколько схем, в т. ч. и следующие две важные схемы:

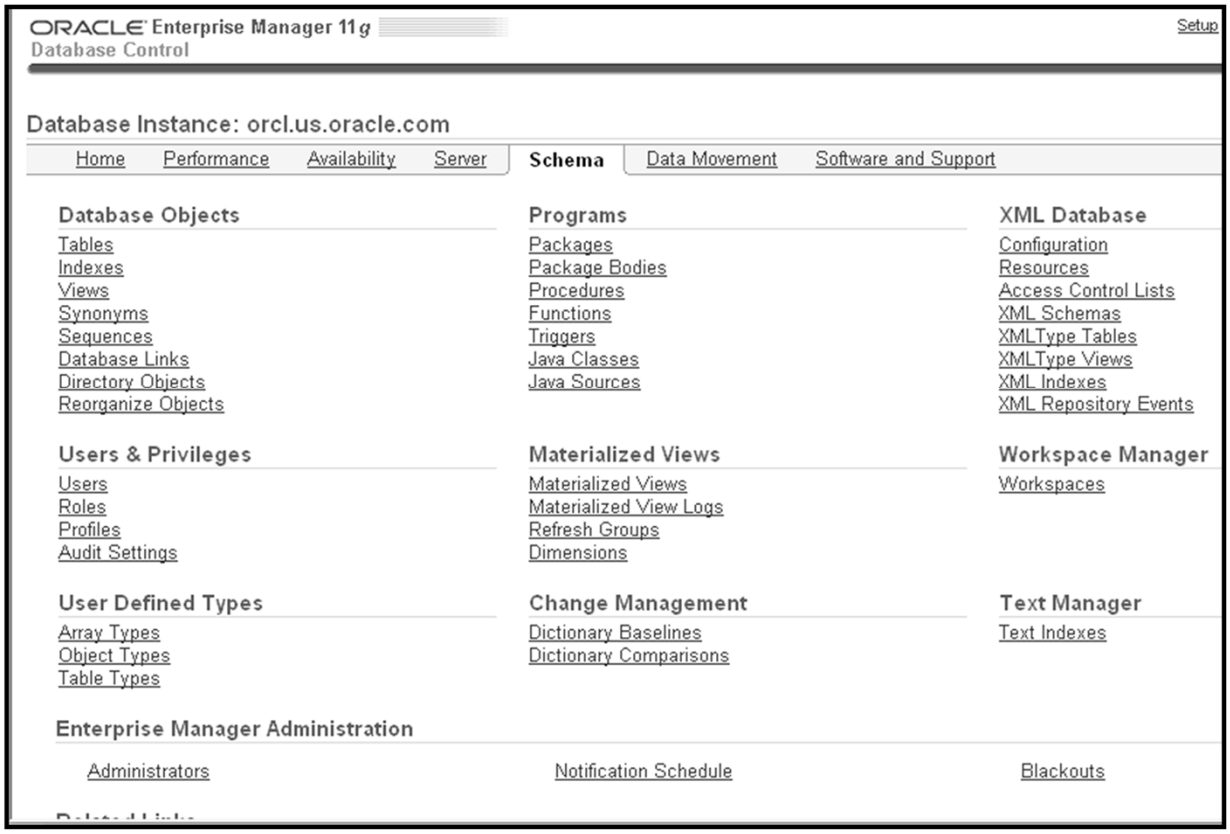
- **Схема `SYS`:** содержит словарь данных, о котором было рассказано в уроке «Администрирование защиты пользователей»;
- **Схема `SYSTEM`:** содержит дополнительные таблицы и представления, в которых хранится административная информация, о чем было рассказано в уроке «Администрирование защиты пользователей».

Что такое схема? (продолжение)

При полной установке БД Oracle также устанавливается несколько образцов схем. Эти схемы предназначены для использования в примерах в документации и учебных программах Oracle. Это набор взаимосвязанных схем, представляющих собой примеры схем различного уровня сложности. В него входят:

- **HR:** несложная схема «Отдел кадров» (Human Resources), используемая для объяснения основных тем. Расширенная версия этой схемы поддерживает демонстрационные примеры Oracle Internet Directory.
- **OE:** схема «Регистрация заказов» (Order Entry), относящаяся к схемам средней сложности. В ней хранятся данные множества типов. Подсхема OC (Online Catalog – интернет-каталог) представляет собой коллекцию объектов объектно-реляционной БД, созданных в схеме OE.
- **PM:** схема «Реклама продукции» (Product Media), предназначенная для хранения данных мультимедийных типов;
- **QS:** схема «Очередь поставок» (Queued Shipping), содержащая набор схем, которые используются для демонстрации возможностей расширенных очередей Oracle;
- **SH:** схема «Журнал продаж» (Sales History), делающая возможными демонстрации с большими объемами данных. Расширенная версия схемы обеспечивает поддержку расширенной аналитической обработки.

Доступ к объектам схем



ORACLE

8 - 5 © Oracle, 2007. Все права защищены.

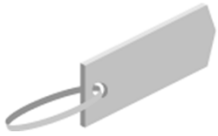
Доступ к объектам схем

Быстро получить доступ ко многим типам объектов схемы можно на странице «Schema» (Схема).

После выбора любой из ссылок отображается страница «Results» (Результаты). В области «Search» (Поиск) на этой странице можно ввести имя схемы и объекта для поиска определенного объекта. Кроме того, можно искать объекты разного типа. Для этого в области «Search» (Поиск) следует выбрать нужный тип в раскрывающемся списке. Этот список содержит дополнительные типы объектов, которые не отображаются на странице «Schema» (Схема) в виде ссылок.

Присвоение имен объектам БД

- Длина имени должна составлять от 1 до 30 байт, за исключением следующих имен:
  - длина имени базы данных не должна превышать 8 байт;
  - длина ссылок на БД не должна превышать 128 байт.
- Нельзя использовать зарезервированные слова Oracle как имена без кавычек.
- Имена без кавычек должны начинаться с буквы из кодовой таблицы базы данных.
- Использование имен в кавычках не рекомендуется.



ORACLE

8 - 6 © Oracle, 2007. Все права защищены.

Присвоение имен объектам БД

Имя объекта базы данных можно взять в двойные кавычки (“”). Однако при этом нарушается несколько правил присвоения имен, приведенных выше. Использовать такие имена объектов не рекомендуется, так как каждая ссылка на такой объект должна будет содержать кавычки вокруг имени. Например, если таблице присвоить имя “`Local Temp`”, то в запросе его следует вводить так:

```
SQL>  select * from "Local Temp";
TEMP_DATE    LO_TEMP    HI_TEMP
--------- ---------- ----------
01-DEC-03         30         41
```

Если ввести имя в неверном регистре, будет выведено сообщение об ошибке:

```
SQL> select * from "local temp";
select * from "local temp"
              *
ОШИБКА в строке 1:
ORA-00942: таблица или представление не существует
```

Имена без кавычек хранятся в верхнем регистре и не чувствительны к регистру. При обработке SQL-оператора все символы имени без кавычек преобразовываются в символы верхнего регистра.

Присвоение имен объектам БД (продолжение)

Идентификаторы без кавычек могут содержать только буквы из кодовой таблицы базы данных, символы подчеркивания (_), знаки доллара ($) и символы решетки (#). Имя связи с базой данных также может содержать точки (.) и символы «at» (@). Настоятельно не рекомендуется использовать в именах идентификаторов символы $ и #.

Идентификаторы в кавычках могут содержать любые символы и знаки препинания, а также пробелы. Тем не менее, ни имена с кавычками, ни без, не могут содержать двойные кавычки.

Определение типов данных в таблице

Общие типы данных:

- **CHAR (*размер* [BYTE|CHAR])**: символьные данные фиксированной длины, имеющие ***размер*** в байтах или символах;
- **VARCHAR2 (*размер* [BYTE|CHAR])**: символьная строка переменной длины с максимальной длиной, имеющей ***размер*** в байтах или символах;
- **DATE**: действительная дата за период от 1 января 4712 г. до н. э. до 31 декабря 9999 г. н. э.;
- **NUMBER (*p*, *s*)**: число с точностью ***p*** и порядком ***s***.

ORACLE

© Oracle, 2007. Все права защищены.

Определение типов данных в таблице

При создании таблицы для каждого ее столбца указывается тип данных. При создании процедуры или функции указывается тип данных всех аргументов. Эти типы данных определяют диапазон значений аргументов или значений, которые могут храниться в столбце.

В БД Oracle используются следующие стандартные типы данных:

- **CHAR:** символьные данные фиксированной длины *размер* байтов или символов. Максимальный размер составляет 2000 байтов или символов, минимальный – 1 байт (используется по умолчанию).
  - BYTE указывает, что в столбце используется семантика размера в байтах (то есть длина значения вычисляется в байтах);
  - CHAR указывает, что в столбце используется семантика размера в символах (то есть строки обрабатываются как последовательности символов).
- **VARCHAR2:** символьная строка переменной длины с максимальной длиной *размер* байтов или символов. Максимальный размер составляет 4000 байтов. Размер VARCHAR2 указывать обязательно.
- **DATE:** действительная дата за период от 1 января 4712 г. до н. э. до 31 декабря 9999 г. н. э. Кроме даты также хранится время (часы, минуты и секунды).
- **NUMBER:** Число с точностью *p* и порядком *s*. Точность может принимать значения от 1 до 38, порядок – от 84 до 127.

Определение типов данных в таблице (продолжение)

- **BINARY_FLOAT:** 32-битное число с плавающей запятой. Для этого типа данных используется 5 байт, включая байт длины.
- **BINARY_DOUBLE:** 64-битное число с плавающей запятой. Для этого типа данных требуется 9 байт.
- **FLOAT(*p*):** тип данных Американского национального института стандартов (ANSI). Тип данных `FLOAT` определяет число с плавающей запятой и двоичной точностью *p*. По умолчанию для этого типа данных используется двоичная точность 126 или десятичная точность 38.
- **INTEGER:** аналог типа `NUMBER(p,0)`;
- **NCHAR(*длина*):** тип данных только для Юникода. При создании таблицы со столбцом типа `NCHAR` длина столбца задается в символах. Национальная кодовая таблица задается при создании базы данных. Максимальная длина столбца задается на основании определения национальной кодовой таблицы. Спецификация ширины типа данных `NCHAR` относится к количеству символов. Максимальный размер столбца составляет 2000 байт. Если в столбец добавляется значение, длина которого меньше длины столбца, оно дополняется пробелами до полной длины столбца. В столбец типа `NCHAR` нельзя добавить значение типа `CHAR`, и наоборот.
- **NVARCHAR2(*размер* [BYTE|CHAR]):** тип данных только для Юникода. Этот тип данных подобен `NCHAR`, только максимальная длина составляет 4000 байт, и значения пробелами не дополняются.
- **LONG:** Символьные данные переменной длины до 2 Гбайт (или 231 – 1 байт). Тип данных `LONG` исключен, вместо него следует использовать тип данных LOB (массивный объект).
- **LONG RAW:** неформатированные двоичные данные переменной длины до 2 Гбайт;
- **RAW(*размер*):** неформатированные двоичные данные длины, имеющей *размер* байт. Максимальный размер составляет 2000 байт. Размер `RAW` указывать обязательно.
- **ROWID:** строка в кодировке Base-64 для хранения уникального адреса строки в БД. Этот тип данных в основном используется для значений, возвращаемых из псевдостолбца `ROWID`.
- **UROWID:** строка в кодировке Base-64 для хранения логического адреса строки в индексной таблице. Дополнительный размер аналогичен размеру столбца типа `UROWID`. Максимальный размер составляет 4000 байт, и он используется по умолчанию. В столбце типа `UROWID` содержится код типа в 1 байте, за которым следует идентификатор строки соответствующего типа. Используются три типа – физический идентификатор строки, логический идентификатор строки и внешний идентификатор строки (используется при выборе столбца по идентификатору строки через dblink посредством шлюза гетерогенной службы).
- **BLOB:** двоичный массивный объект;
- **CLOB:** символьный массивный объект, содержащий однобайтовые или многобайтовые символы. Для обоих типов символов поддерживаются кодовые таблицы постоянной и переменной длины, а также используется кодовая таблица базы данных `CHAR`.

Примечание. Для односимвольных кодовых таблиц столбцы, определенные семантикой размера в символах или в байтах, обычно не отличаются. Семантика размера в символах удобна для определения многобайтовых строк переменной длины, так как упрощается определение фактических требований к длине данных для хранилищ данных. Например, в базе данных типа Юникод (`UTF8`) необходимо определить столбец типа `VARCHAR2`, в котором можно будет хранить до пяти китайских символов с пятью английскими. При использовании семантики размера в байтах потребуется (5 * 3 байт) + (1 * 5 байт) = 20 байт.

При использовании семантики размера в символах для столбца потребуется 10 символов.

Определение типов данных в таблице (продолжение)

- **NCLOB:** символьный массивный объект, содержащий символы Юникода.
  Поддерживаются кодовые таблицы постоянной и переменной длины (в обоих случаях используется кодовая таблица базы данных NCHAR). Этот тип данных используется для хранения данных национальной кодовой таблицы.
  Примечание. Максимальный размер любых данных типа LOB (BLOB, CLOB и NCLOB) равен (4 Гбайт – 1) * (значение CHUNK).
  CHUNK – это дополнительный атрибут, который можно задать при определении LOB. Атрибут CHUNK задает количество байтов, выделяемых для обработки данных LOB. Если размер данных не кратен размеру блока данных БД, он округляется в байтах до следующего кратного числа.
  К примеру, размер блока данных БД равен 2048, размер CHUNK равен 2050, в БД выделяется 4096 байт (2 блока). Максимальное значение составит 32768 (32 Кбайт), это наибольший допустимый размер блока данных в БД Oracle. По умолчанию размер CHUNK равен размеру одного блока БД Oracle.
- **BFILE:** используется для хранения локатора для большого двоичного файла, который хранится за пределами базы данных. Это обеспечивает доступ для ввода/вывода потока байтов внешних данных LOB, хранящихся на сервере базы данных. Максимальный размер составляет 4 Гбайт.
- **TIMESTAMP(*точность_в_долях_секунды*):** определяет значения года, месяца и дня для даты, а также часы, минуты и секунды для времени. Аргумент *точность_в_долях_секунды* задает количество разрядов в дробной части числа секунд. Допустимые значения от 0 до 9. По умолчанию используется значение 6.

Полный список стандартных типов данных можно изучить в руководстве *Oracle Database SQL Reference*.

Создание и изменение таблиц

Database Instance: orcl.us.oracle.com > Tables > Logged in As SYS

Create Table

Show SQL | Cancel | OK

General | Constraints | Storage | Options | Partitions

Ввод имени таблицы и схемы.

* Name jobs

Schema SHOPOWNER

Tablespace USERS Estimate Table Size

Organization **Standard, Heap Organized**

Wallet Status **CLOSED**

Определение имен, типов данных и длины столбцов.

Define Using Column Specification

Set Default LOB Attributes

Advanced Attributes | Delete | Insert Column: Abstract Data Type | Insert | Encryption Options

Select		Name	Data Type	Size	Scale	Not NULL	Default Value	Encrypted
		job_id	NUMBER	5				
		job_title	VARCHAR2	30				
		min_salary	NUMBER	6				
		max_salary	NUMBER	6				

ORACLE

8 - 11 © Oracle, 2007. Все права защищены.

Создание и изменение таблиц

Таблица представляет собой основную единицу хранения данных в БД Oracle. В каждой таблице содержатся столбцы и строки, в которых хранятся все доступные пользователям данные.

Создание таблицы

Чтобы создать таблицу с помощью Enterprise Manager, выполните следующие действия:

1. На странице «Schema» (Схема) щелкните пункт «Tables» (Таблицы). На экране отобразится страница «Tables» (Таблицы).
2. Если известно имя схемы, введите его полностью или частично в области «Search» (Поиск) в поле «Schema» (Схема). Если же имя схемы неизвестно, возле поля «Schema» (Схема) щелкните значок фонарика. На экране отобразится окно «Search and Select: Schema» (Поиск и выбор: схема). Можно просмотреть имена схем и выбрать из них нужную.
3. Нажмите кнопку «Create» (Создать). На экране отобразится страница «Create Table: Organization» (Создание таблицы: организация).
4. Нажмите кнопку «Continue» (Продолжить), чтобы принять стандартные настройки «Standard, Heap Organized» (Стандартная, с динамической организацией). На экране отобразится страница «Create Table» (Создание таблицы).
5. В поле «Name» (Имя) введите имя таблицы.
6. В поле «Schema» (Схема) введите имя схемы или щелкните значок фонарика, чтобы

найти необходимую схему.

Создание и изменение таблиц

```
CREATE TABLE SHOPOWNER.JOBS (
   Job_id NUMBER(5),
   Job_title VARCHAR2(30),
   MIN_SALARY NUMBER(6),
   MAX_SALARY NUMBER(6)
   )
   TABLESPACE USERS;
```

```
ALTER TABLE SHOPOWNER.JOBS add bonus NUMBER(6);
```

ORACLE

© Oracle, 2007. Все права защищены.

Создание и изменение таблиц (продолжение)

7. В поле «Tablespace» (Табличное пространство) введите имя табличного пространства или щелкните значок фонарика, чтобы выполнить поиск имени.
8. В области «Columns» (Столбцы) введите имена и тип данных столбцов.
9. Нажмите кнопку «OK». На экране отобразится сообщение об успешном создании таблицы.

Изменение таблицы

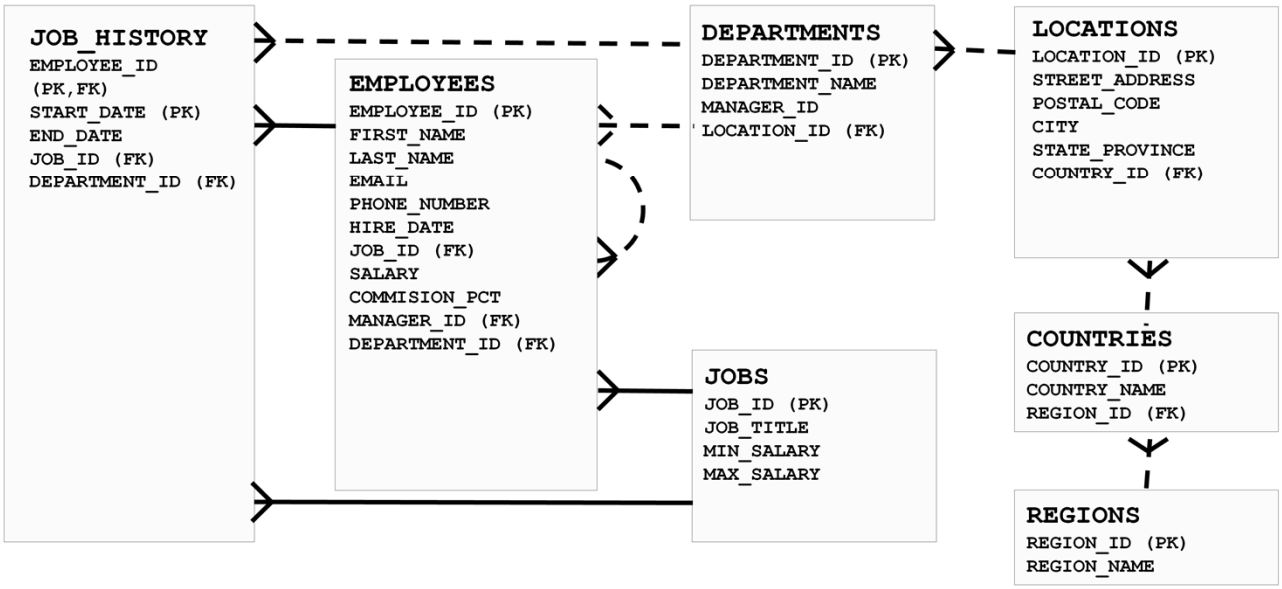
Изменить таблицу можно с помощью Enterprise Manager. Например, чтобы добавить в таблицу столбец, выполните следующие действия:

1. На странице «Tables» (Таблицы) в списке результатов выберите таблицу и нажмите кнопку «Edit» (Изменить).
2. На странице «Edit Table» (Редактировать таблицу) нажмите кнопку «Add 5 Table Columns» (Добавить 5 столбцов в таблицу). На экране отобразится список столбцов, доступный для редактирования.
3. Введите имя, тип данных и размер нового столбца. После этого нажмите кнопку «Apply» (Применить). На экране отобразится сообщение об успешном изменении таблицы.

Также можно создавать таблицы с помощью SQL-операторов. На слайде показан пример, в котором в схеме `SHOPOWNER` создается таблица `JOBS`, создаваемая в табличном пространстве `USERS`.

Понимание целостности данных

Схема
> Ограничения
Индексы
Представления
Последовательности
Врем. таблицы
Словарь данных



ORACLE

8 - 13 © Oracle, 2007. Все права защищены.

Понимание целостности данных

Для недопущения введения в столбцы тех или иных значений можно использовать следующие ограничения:

- **NOT NULL:** по умолчанию все столбцы таблицы могут принимать неопределенные значения. Слово *null* означает отсутствие значения. Ограничение `NOT NULL` указывает, что столбец таблицы не должен содержать неопределенных значений. Например, можно определить ограничение `NOT NULL`, чтобы указать, что ввод значений в столбец `LAST_NAME` обязателен для всех строк таблицы `EMPLOYEES`.
- **Ключ UNIQUE:** ограничение целостности для ключа `UNIQUE` указывает, что все значения в столбце или наборе столбцов (ключе) должны быть уникальными, то есть ни одна пара строк не должна содержать одинаковых значений в этом столбце или наборе столбцов. Например, ограничение ключа `UNIQUE` определено для столбца `DEPARTMENT_NAME` в таблице `DEPARTMENTS`, чтобы запретить строки с одинаковыми названиями отделов. За исключением отдельных случаев это задается с помощью уникального индекса.
- **PRIMARY KEY:** любая таблица в БД может иметь не более одного ограничения `PRIMARY KEY`. Значения из группы одного или нескольких столбцов, для которых определено данное ограничение, образуют уникальный идентификатор строки. Фактически, все строки именуются по значению первичного ключа.

Понимание целостности данных (продолжение)

Реализация ограничения целостности PRIMARY KEY в сервере Oracle обеспечивает выполнение двух следующих условий:

- Ни одна пара строк таблицы не содержит одинаковых значений в заданном столбце или группе столбцов.
- Столбец первичных ключей не может содержать неопределенных значений. Таким образом, каждая строка такого столбца должна содержать значение.

  Обычно ограничения PRIMARY KEY накладываются с помощью индексов. Ограничение первичного ключа, созданное для столбца DEPARTMENT_ID в таблице DEPARTMENTS, наложено посредством явного создания:
  - уникального индекса для данного столбца;
  - ограничения NOT NULL для данного столбца.
- **Ссылочные ограничения целостности:** разные таблицы в реляционной базе данных могут быть связаны общими столбцами, поэтому необходимо соблюдать правила отношения столбцов. Ссылочные правила целостности гарантируют сохранение этих отношений.

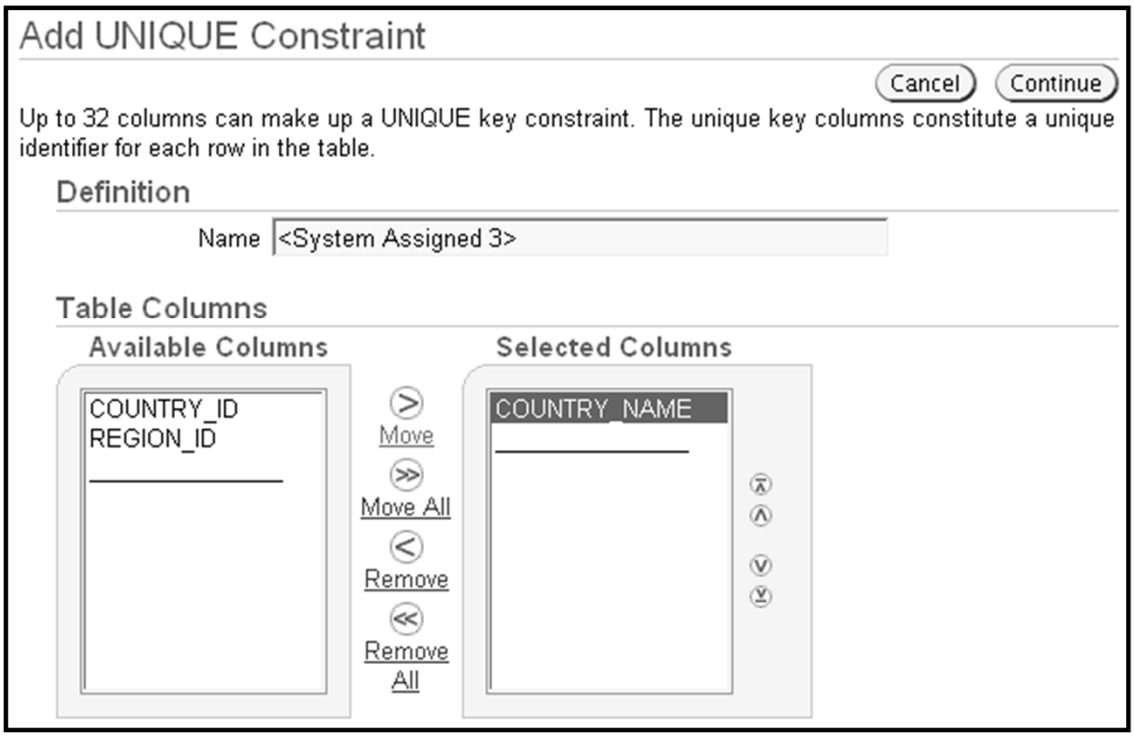
  Ссылочные ограничения целостности накладывают следующее требование: для всех строк таблицы значение во внешнем ключе должно совпадать со значением в первичном ключе.

  Например, внешний ключ определен для столбца DEPARTMENT_ID таблицы EMPLOYEES. Это гарантирует, что все значения в столбце совпадают со значениями в первичном ключе таблицы DEPARTMENTS. Следовательно, в столбце DEPARTMENT_ID таблицы EMPLOYEES отсутствуют неправильные номера отделов.

  Еще один тип ссылочного ограничения целостности называется рефлексивным ограничением ссылочной целостности. При этом внешний ключ ссылается на родительский ключ в той же таблице.
- **Ограничения CHECK:** ограничение целостности CHECK для столбца или набора столбцов указывает, что для каждой строки таблицы должно выполняться или иметь неизвестное значение какое-либо заданное условие.

Если при выполнении DML-оператора проверки условия ограничения получен результат FALSE, выполняется откат оператора, если наложено ограничение IMMEDIATE. Если ограничение является отложенным (DEFERRED), откат производится при фиксации, а не выполнении DML.

Определение ограничений



© Oracle, 2007. Все права защищены.

Определение ограничений

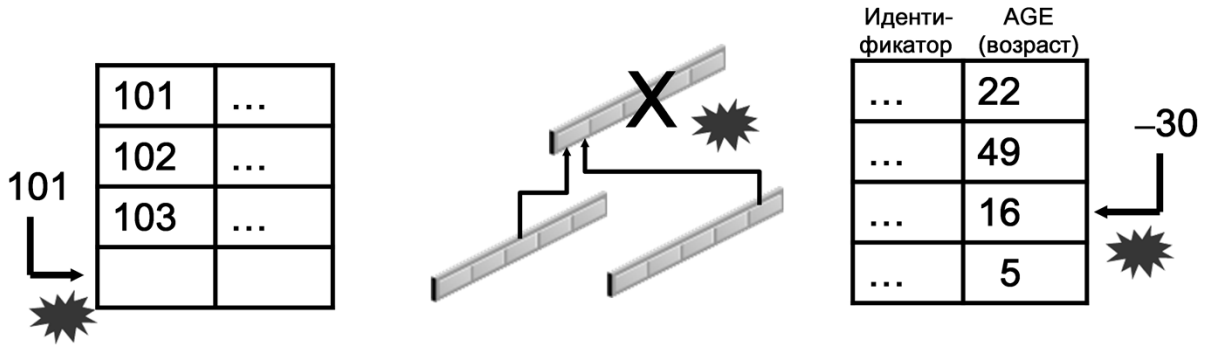
Чтобы добавить ограничение для таблицы с помощью Enterprise Manager:

1. На странице «Tables» (Таблицы) выберите таблицу и нажмите кнопку «Edit» (Редактировать).
2. Щелкните на пункте «Constraints» (Ограничения). На странице «Constraints» (Ограничения) отображаются все ограничения, определенные для выбранной таблицы.
3. Из раскрывающегося списка выберите тип добавляемого ограничения, после чего нажмите кнопку «Add» (Добавить).
4. Введите необходимую информацию для типа добавляемого ограничения. Нажмите кнопку «OK».

Нарушение ограничений

Примеры нарушений ограничения:

- добавление повторного значения первичного ключа;
- удаление родительского элемента дочерней строки при наличии ссылочного ограничения целостности;
- обновление таблицы с использованием значения, выходящего за пределы ограничения CHECK.



© Oracle, 2007. Все права защищены.

Нарушение ограничений

Нарушение ограничения происходит, если переданный DML-оператор не совместим с ограничением. Нарушения ограничения могут обуславливаться разными условиями. Вот некоторые из них:

- **Неуникальность:** попытка хранения одинаковых значений в столбце с ограничением UNIQUE (например, если столбец является первичным ключом или используется уникальный индекс);
- **Целостность ссылочных данных:** нарушение правила, согласно которому у каждой дочерней строки должна быть родительская строка;
- **Проверка:** попытка сохранить в столбце значение, противоречащее правилам, определенным для столбца. Например, для столбца `AGE` задано ограничение CHECK, согласно которому все значения столбца должны быть положительными.

Состояния ограничений

Чтобы проще разрешать ситуации, когда данные временно могут нарушать ограничение, можно использовать различные состояния ограничений. Ограничение целостности можно включить (`ENABLE`) или выключить (`DISABLE`).

Если ограничение включено, проверка данных происходит при вводе данных или обновлении БД. Данные, противоречащие правилам ограничения, не вводятся.

Если ограничение отключено, в базу данных могут вводиться противоречащие ограничению данные.

Ограничение целостности может находиться в одном из следующих состояний:

- `DISABLE NOVALIDATE`
- `DISABLE VALIDATE`
- `ENABLE NOVALIDATE`
- `ENABLE VALIDATE`

Состояния ограничений (продолжение)

DISABLE NOVALIDATE: новые или уже существующие данные могут противоречить ограничению, если они не проверены. Этим часто пользуются при обработке данных из уже проверенного источника в режиме таблицы только для чтения, вследствие чего новые данные в таблицу не попадают. Состояние `NOVALIDATE` используется для хранилищ данных, в которых данные уже прошли проверку. Поэтому повторная проверка не нужна, что экономит время обработки.

DISABLE VALIDATE: при данном состоянии ограничения изменение столбцов, на которых оно наложено, запрещено, так как это нарушит целостность данных – уже существующие данные будут проверены, а новые будут вводиться без проверки. Это состояние часто используют, когда существующие данные нужно проверить, но не изменить, а индекс для производительности не нужен.

ENABLE NOVALIDATE: новые данные соответствуют ограничению, однако уже существующие данные находятся в неизвестном состоянии. Это состояние часто используется, когда точно известно, что таблица содержит обработанные и проверенные данные, не требующие проверки. При этом все новые вводимые данные не должны противоречить ограничению.

ENABLE VALIDATE: ограничению не должны противоречить ни новые, ни существующие данные. Это самое распространенное состояние ограничения, используемое по умолчанию.

Проверка ограничений

Ограничения проверяются при следующих событиях:

- исполнение оператора (для неотложенных ограничений);
- `COMMIT` (для отложенных ограничений).

Ситуация: после DML-оператора выполняется `COMMIT`.

1. Проверка неотложенных ограничений
2. Выполнение `COMMIT`
3. Проверка отложенных ограничений
4. Завершение `COMMIT`

ORACLE

© Oracle, 2007. Все права защищены.

Проверка ограничений

Проверку соответствия данных ограничению можно отложить до завершения транзакции.

Неотложенные ограничения, также называемые *немедленными ограничениями*, применяются после выполнения каждого DML-оператора. При нарушении ограничения выполняется откат оператора. Если ограничение приводит к такой операции, как каскадное удаление (`delete cascade`), эта операция выполняется как часть вызвавшего ее оператора. Если ограничение определено как неотложенное, его нельзя сделать отложенным При использовании неотложенных ограничений для ограничений первичного и уникального ключей требуются уникальные индексы. Если же для одного или более столбцов уже используется уникальный индекс, создать ограничение будет невозможно, так как эти индексы нельзя использовать для уникального или первичного ключа.

Отложенные ограничения проверяются только после фиксации транзакции.
Если при этом нарушается ограничение, выполняется откат всей транзакции. Такие ограничения удобнее всего использовать, когда при отношении внешнего ключа родительские и дочерние строки вводятся одновременно, как в примере с системой регистрации заказов, где заказ и его позиции вводятся одновременно. При использовании отложенных ограничений для ограничений первичного и уникального ключей требуются неуникальные индексы. Если же для одного или более столбцов уже используется уникальный индекс, создать ограничение будет невозможно, так как эти индексы не являются отложенными.

Проверка ограничений (продолжение)

Определить отложенное ограничение можно одним из предложенных способов:

- `Initially immediate` по умолчанию действует как немедленное ограничение, если только явно не определено обратное;
- `Initially deferred` указывает, что по умолчанию ограничение применяется только после выполнения транзакции.

Примечание. Если для столбца уже используется подходящий индекс, он используется для ограничения. Для первичного и уникального ключей дополнительный индекс не требуется.

Создание ограничений с помощью SQL: примеры

a
```
ALTER TABLE countries
ADD (UNIQUE(country_name) ENABLE NOVALIDATE);
```

b
```
ALTER TABLE shopowner.jobs ADD CONSTRAINT job_pk PRIMARY
KEY (job_id);
```

c
```
CREATE TABLE emp (emp_no NUMBER PRIMARY KEY,Last_name
VARCHAR2(30), first_name VARCHAR2(30),  dept_no NUMBER,
Mgr_no NUMBER, hire_date date,salary NUMBER,
CONSTRAINT Mgr_FK FOREIGN KEY (mgr_no) REFERENCES
emp(emp_no), CONSTRAINT ck1 CHECK (salary > 0));
```

ORACLE

8 - 21 © Oracle, 2007. Все права защищены.

Создание ограничений с помощью SQL: примеры

а. После выполнения оператора любые добавляемые или обновляемые данные в таблице COUNTRIES должны содержать уникальное значение COUNTRY_NAME. Однако при этом после выполнения оператора в таблице могут храниться старые значения COUNTRY_NAME, которые не являются уникальными. Ключевое слово NOVALIDATE указывает, что их нужно игнорировать. Ограничение применяется только к новым строкам.

б. Данный оператор добавляет первичный ключ в таблицу JOBS. Ограничение называется JOB_PK, первичным ключом является столбец JOB_ID.

в. Оператор определяет ограничения при создании таблицы, а не после создания посредством оператора ALTER TABLE. Ограничение MGR_FK указывает, что значения столбца MGR_NO должны присутствовать в столбце первичного ключа таблицы. Ограничение CK1 указывает, что значения SALARY должны быть больше нуля.

Создание ограничений с помощью SQL: примеры (продолжение)

При нарушении ограничения на экран выводится сообщение об ошибке:

```
 INSERT INTO emp
(Select employee_id , last_name, first_name,department_id,
manager_id, hire_date, salary FROM HR.employees where
department_id =30);
  2  INSERT INTO emp
*
ОШИБКА в строке 1:
ORA-02291: integrity constraint (SYS.MGR_FK) violated - parent
key not found
```

Примечание. У каждого ограничения есть имя. Если имя не указано при создании ограничения, ему присваивается системное имя, которое начинается на `SYS_`.

Просмотр столбцов таблицы

View Table: HR.DEPARTMENTS

Actions Create Like Go Edit OK

General

Name DEPARTMENTS
Schema HR
Tablespace EXAMPLE
Organization Standard, Heap Organized

Columns

	Name	Data Type	Size	Scale	Not NULL	Default Value	Encrypted
	DEPARTMENT_ID	NUMBER	4		☑		☐
	DEPARTMENT_NAME	VARCHAR2	30		☑		☐
	MANAGER_ID	NUMBER	6		☐		☐
	LOCATION_ID	NUMBER	4		☐		☐

Indicates a Primary Key column
Indicates a Unique Key column

Constraints

Previous 1-4 of

Name	Type	Table Columns	Disabled	Deferrable	Initially Deferred	Validate	RELY	Check Condition	Referenced Schema	Referenced Table	Referenced Table Columns
DEPT_ID_PK	PRIMARY	DEPARTMENT_ID	NO	NO	NO	YES	NO				
DEPT_LOC_FK	FOREIGN	LOCATION_ID	NO	NO	NO	YES	NO		HR	LOCATIONS	LOCATION_ID
DEPT_MGR_FK	FOREIGN	MANAGER_ID	NO	NO	NO	YES	NO		HR	EMPLOYEES	EMPLOYEE_ID
DEPT_NAME_NN	CHECK	DEPARTMENT_NAME	NO	NO	NO	YES	NO	"DEPARTMENT_NAME" IS NOT NULL			

Indexes

Number of Indexes 2

Schema.Index	Indexed Columns	Column Position

ORACLE

8 - 23 © Oracle, 2007. Все права защищены.

Просмотр столбцов таблицы

Чтобы просмотреть атрибуты таблицы с помощью Enterprise Manager, выполните следующие действия:

1. На странице «Database Administration» (Администрирование БД) в области «Schema» (Схема) щелкните ссылку «Tables» (Таблицы).
2. Из списка «Results» (Результаты) выберите таблицу и нажмите кнопку «View» (Просмотреть), чтобы просмотреть атрибуты таблицы.

Также атрибуты таблицы можно просмотреть с помощью команды `DESCRIBE` утилиты SQL*Plus:

```
SQL> desc hr.departments
 Name                      Null?    Type
 ------------------------- -------- --------
 DEPARTMENT_ID             NOT NULL NUMBER(4)
 DEPARTMENT_NAME           NOT NULL VARCHAR2(30)
 MANAGER_ID                         NUMBER(6)
 LOCATION_ID                        NUMBER(4)
```

Просмотр содержимого таблицы

View Data for Table: HR.DEPARTMENTS

Refine Query | OK

Query
```
SELECT "DEPARTMENT_ID", "DEPARTMENT_NAME", "MANAGER_ID", "LOCATION_ID"
FROM "HR"."DEPARTMENTS"
```

Result — Previous 1-25 of 27 Next 2

DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	LOCATION_ID
10	Administration	200	1700
20	Marketing	201	1800
30	Purchasing	114	1700
40	Human Resources	203	2400
50	Shipping	121	1500
60	IT	103	1400
70	Public Relations	204	2700
80	Sales	145	2500
90	Executive	100	1700
100	Finance	108	1700
110	Accounting	205	1700

Done — edtdr8p1:1158

ORACLE

8 - 24 © Oracle, 2007. Все права защищены.

Просмотр содержимого таблицы

Чтобы просмотреть строки таблицы с помощью Enterprise Manager:

1. На странице «Tables» (Таблицы) выберите таблицу.
2. В меню «Actions» (Действия) выберите пункт «View Data» (Представление данных). После этого нажмите кнопку «Go» (Перейти).

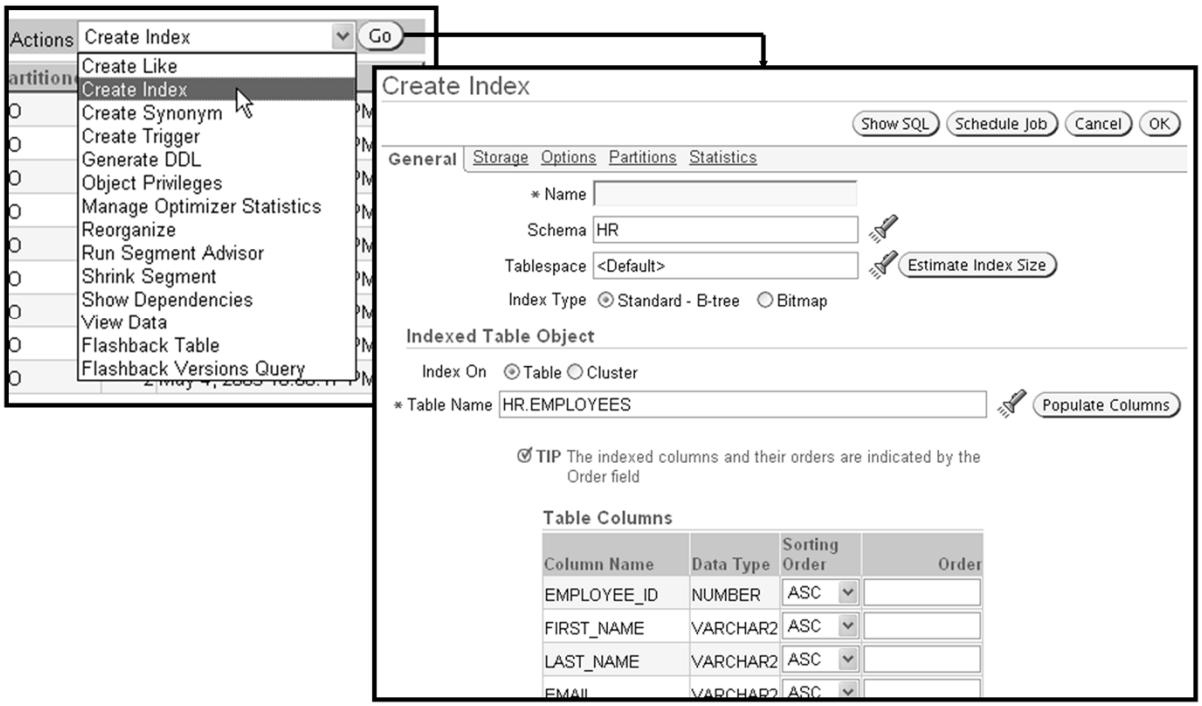
На экране отобразится страница «View Data» (Представление данных таблицы). Данные из строк таблицы отображаются в области «Result» (Результат). В окне «Query» (Запрос) отображается SQL-запрос, который был выполнен для получения результатов. На этой странице можно щелкнуть имя любого столбца и отсортировать данные в нем по возрастанию или убыванию. Если необходимо изменить запрос, нажмите кнопку «Refine Query» (Уточнить запрос). На странице «Refine Query for Table» (Уточнение запроса для таблицы) можно выбрать столбцы, которые нужно отобразить, и указать фразу `WHERE` для ограничения результатов выполнения SQL-оператора.

Подробнее о фразе `WHERE` в SQL-операторах описано в руководстве *Oracle Database SQL Reference*.

Также можно выполнить запрос в строке ввода SQL:

```
SQL> SELECT department_id , department_name, manager_id,
manager_name
FROM departments;
```

Операции с таблицами



ORACLE

8 - 25 © Oracle, 2007. Все права защищены.

Операции с таблицами

Таблицу можно выбрать, после чего выполнять с ней определенные операции. Вот некоторые из этих операций:

- **Создать подобный:** создание таблицы со структурой, аналогичной выбранной таблице. Имена ограничений необходимо будет изменить. Перед созданием таблицы можно добавить или удалить столбцы, а также внести другие изменения в структуру.
- **Создать индекс:** создание индексов для таблицы;
- **Создать DDL:** создание кода DDL, соответствующего уже существующей таблице. Этот код можно скопировать в текстовый файл и использовать как сценарий или для составления документации.
- **Предоставить полномочия:** по умолчанию право на операции с созданной таблицей имеет только ее владелец. Чтобы другие пользователи могли выполнять операции с таблицей при помощи DML или, возможно, DDL, владелец таблицы должен предоставить соответствующие полномочия.
- **Показать зависимости:** отображение объектов, от которых зависит таблица, или объектов, зависящих от таблицы;
- **Представление данных:** выбор и отображение данных таблицы в режиме только для чтения.

Удаление таблицы

При удалении таблицы удаляются:

- данные;
- структура таблицы;
- триггеры базы данных;
- соответствующие индексы;
- связанные с таблицей объектные полномочия.

```
DROP TABLE hr.employees PURGE;
```

Дополнительно с оператором **DROP TABLE** можно использовать следующие фразы:

- **CASCADE CONSTRAINTS**: зависимые ссылочные ограничения целостности;
- **PURGE**: без возможности восстановления.

ORACLE

8 - 26 © Oracle, 2007. Все права защищены.

Удаление таблицы

Синтаксис:

```
DROP TABLE [схема.] таблица [CASCADE CONSTRAINTS] [PURGE]
```

Команда DROP TABLE удаляет данные, структуру таблицы, связанные с ней объектные полномочия и секции, если они существуют.

Некоторые характеристики DROP TABLE:

- без фразы PURGE определение таблицы, связанные индексы и триггеры помещаются в корзину; данные таблицы не удаляются, однако они недоступны без определения таблицы; если удалить таблицу с помощью Enterprise Manager, фраза PURGE не используется;
- для восстановления объектов схемы из корзины используется команда FLASHBACK TABLE; команда PURGE RECYCLEBIN очищает корзину;
- параметр CASCADE CONSTRAINTS необходим для удаления всех зависящих от таблицы ссылочных ограничений целостности.

Примечание. Если параметр PURGE не используется, пространство, которое занимала таблица и ее индексы, по прежнему вычитается из квоты пользователя в используемом табличном пространстве. То есть пространство по-прежнему считается используемым.

Усечение таблицы

- При усечении таблицы удаляются данные и увеличивается свободное пространство.
- Соответствующие индексы также усекаются.

```
TRUNCATE TABLE hr.employees;
```

ORACLE

8 - 27 © Oracle, 2007. Все права защищены.

Усечение таблицы

Синтаксис:

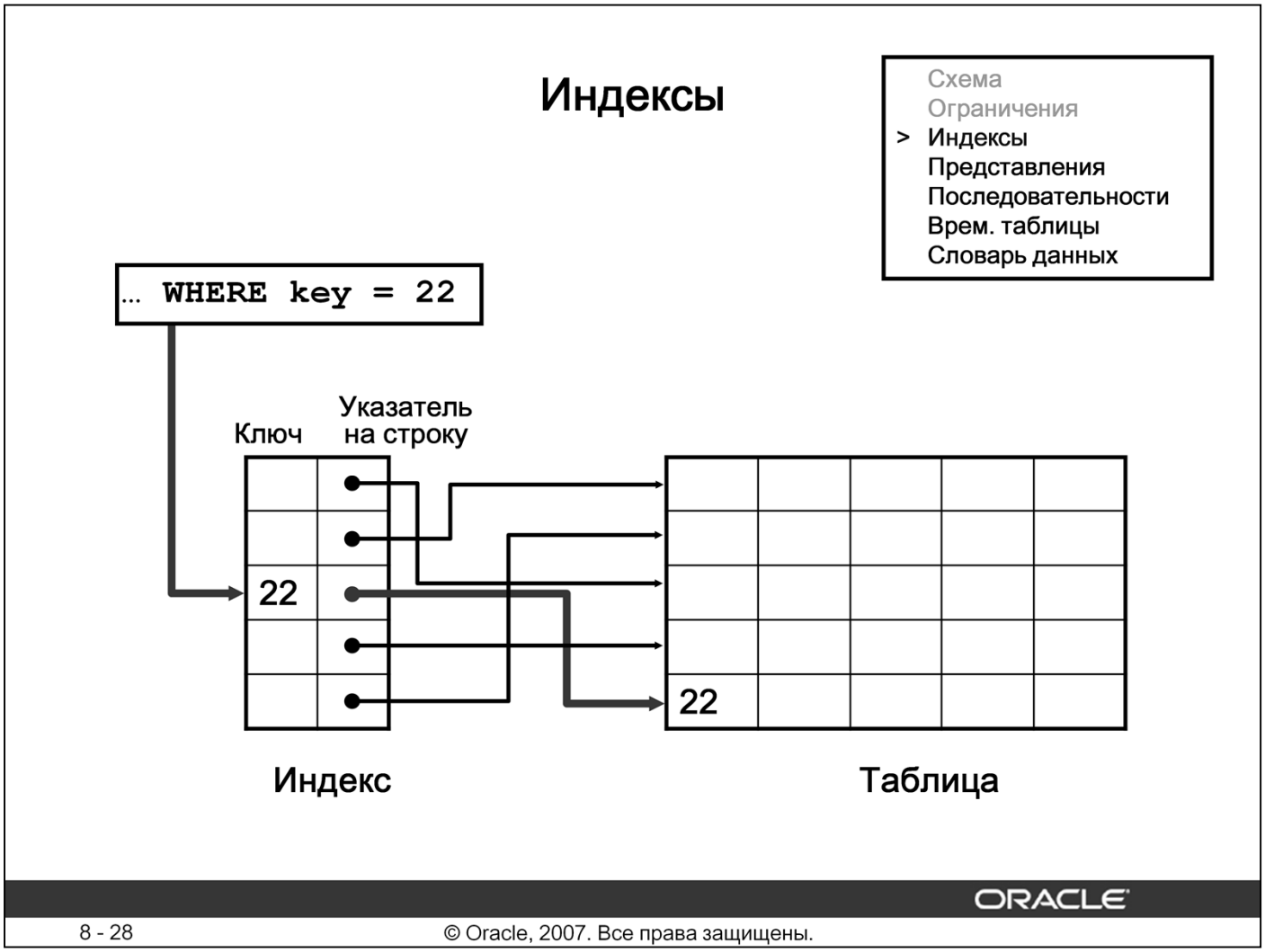
```
TRUNCATE TABLE [схема.] таблица [{DROP | REUSE} STORAGE]
```

Результаты использования команды:

- таблица помечается как пустая, для этого максимальный уровень заполнения (HWM) переносится на начало таблицы, и ее строки становятся недоступными;
- данные отмены операций не создаются, и фиксация команды происходит неявно, так как `TRUNCATE TABLE` является командой DDL.
- соответствующие индексы также усекаются;
- таблицу, на которую ссылается внешний ключ, усечь невозможно;
- триггеры удаления при использовании команды не срабатывают.

Обычно эта операция выполняется в несколько раз быстрее, чем оператор `DELETE`, удаляющий все строки таблицы, ввиду следующих причин:

- БД Oracle сбрасывает HWM таблицы, а не обрабатывает каждую строку, как при операции `DELETE`;
- не создаются данные отмены операций.



Индексы

Индексы – это дополнительные структуры, связанные с таблицами. Они создаются для повышения производительности операций обновления и получения данных. Индекс Oracle позволяет напрямую обратиться к строке с данными.

Индексы можно создать для одного или более столбцов таблицы. Созданный индекс автоматически управляется и используется сервером Oracle. При обновлении данных таблицы, например при добавлении, обновлении или удалении строк, все внесенные изменения автоматически распространяются на все связанные индексы с соблюдением полной прозрачности для пользователей.

Индексы могут повысить производительность разрешения ограничений первичного и внешнего ключа. Если индексы отсутствуют, при каждом выполнении DML-операции с таблицей сканируется вся таблица (полный просмотр таблицы).

Типы индексов

Существует несколько типов индексных структур, в зависимости от требуемых свойств. Два самых распространенных типа:

- Индекс b-дерева
  - тип индекса по умолчанию, имеет форму сбалансированного дерева;
- Битовый индекс:
  - содержит битовый образ каждого неповторяющегося индексированного значения;
  - каждая битовая позиция соответствует строке, которая может содержать или не содержать индексированное значение;
  - лучше всего подходит для столбцов с низкой кардинальностью.

ORACLE

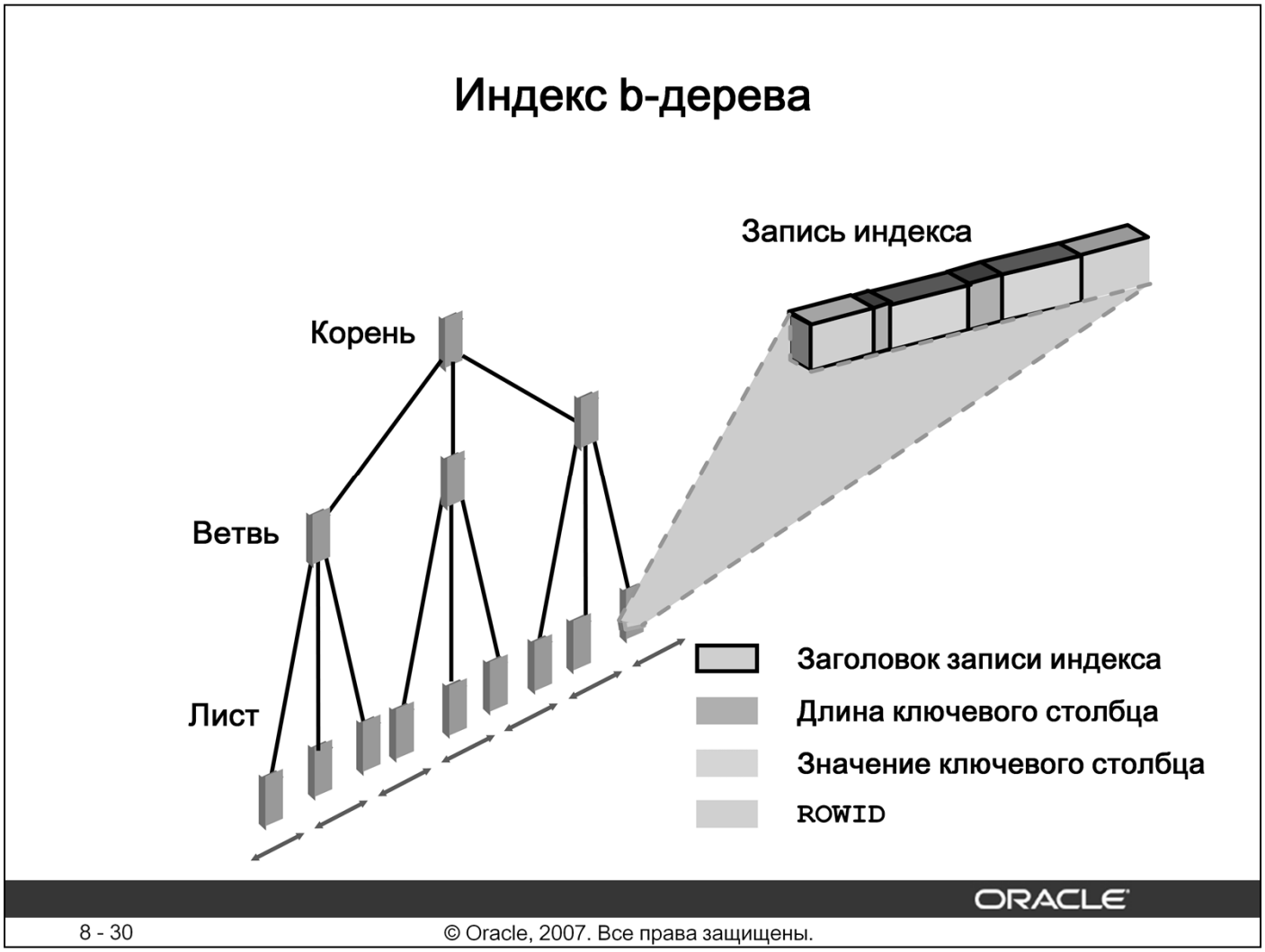
8 - 29 © Oracle, 2007. Все права защищены.

Типы индексов

Ключевые значения индекса b-дерева хранятся в структуре в виде сбалансированного дерева (b-дерева), что делает возможным быстрый двоичный поиск.

Битовый индекс содержит битовый образ каждого неповторяющегося индексированного значения. В каждом битовом образе хранится бит, предназначенный для соответствующей строки индексированной таблицы. Это ускоряет поиск, если в индексированном столбце мало отличающихся значений, ввиду чего он обладает низкой кардинальностью. Битовые индексы следует использовать для столбцов с низкой кардинальностью.

В качестве примера можно взять показатель половой принадлежности, который может принимать исключительно значения «М» и «Ж». Из-за этого для поиска будет использоваться всего два битовых образа. Если бы битовый индекс использовался для столбца телефонных номеров `phone_number`, пришлось бы обрабатывать так много битовых образов, что это фатально сказалось бы на производительности.



Индекс b-дерева

Структура индекса b-дерева

На верхнем уровне индекса находится корень, содержащий элементы с указателями следующего уровня. На следующем уровне располагаются блоки ветвей, которые в свою очередь указывают на следующий уровень индекса. На нижнем уровне находятся листовые узлы, содержащие записи индекса с указателями на строки таблицы. Блоки листьев используют двойные связи, это упрощает сканирование индекса при возрастающем или убывающем порядке ключевых значений.

Формат листовой записи индекса

Запись индекса содержит следующие составляющие:

- **Заголовок индекса:** содержит количество столбцов и информацию о блокировании;
- **Пары значение-длина ключевого столбца:** определяет размер столбца в ключе и значение для столбца (количество таких пар равно максимальному количеству столбцов в индексе);
- **`ROWID`:** идентификатор строки с ключевыми значениями.

Индекс b-дерева (продолжение)

Свойства листовой записи индекса

Для индекса b-дерева в несекционированной таблице:

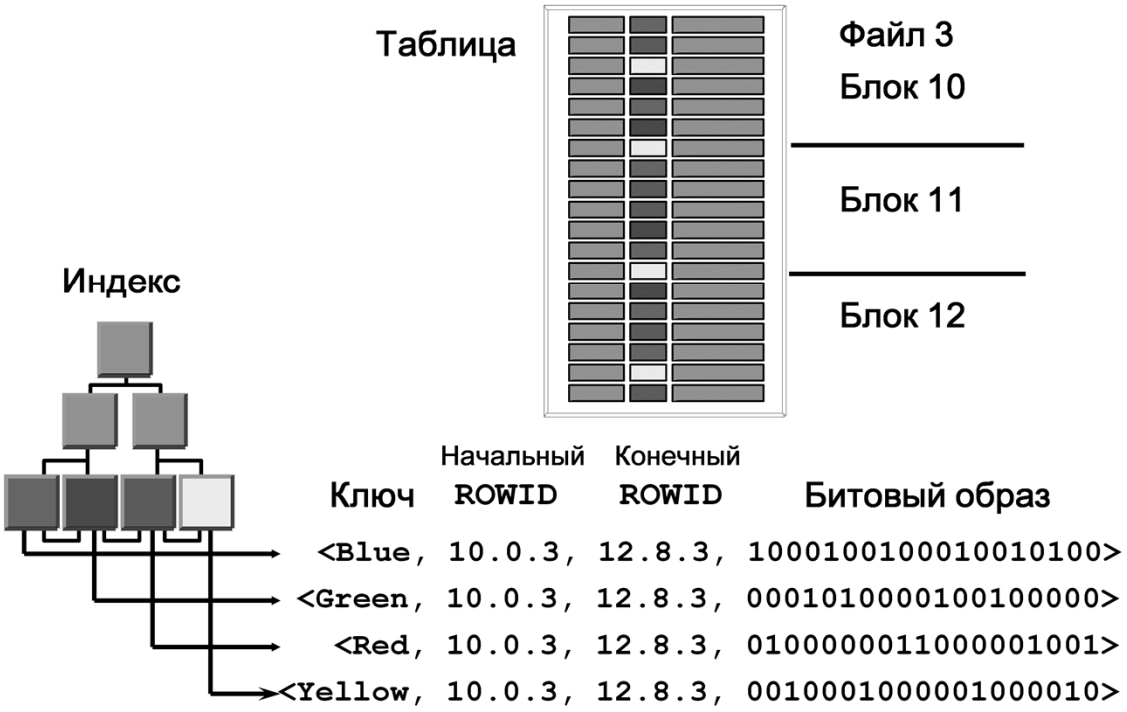
- ключевые значения повторяются, если в нескольких строках хранятся одинаковые ключевые значения и индекс не сжат;
- если для строки все ключевые столбцы содержат значение `NULL`, то соответствующая строке запись индекса отсутствует. Следовательно, фраза `WHERE` с аргументом `NULL` всегда будет выполнять полный просмотр таблицы.
- в качестве указателя на строки таблицы используется ограниченный `ROWID`, поскольку все строки хранятся в одной секции.

Результат операции DML с индексом

При выполнении операции DML с таблицей сервер Oracle обслуживает все индексы. Ниже описано воздействие DML-команды на индекс:

- операция добавления приводит к добавлению записи индекса в соответствующий блок;
- удаление строки вызывает только логическое удаление записи индекса. При этом использовавшееся удаленной строкой пространство доступно для новых последующих листовых записей.
- при обновлении ключевых столбцов в индексе производится логическое удаление и добавление; параметр `PCTFREE` влияет на индекс только при его создании. В блок индекса можно добавить новую запись, даже если в блоке меньше места, чем задано параметром `PCTFREE`.

Битовые индексы



© Oracle, 2007. Все права защищены.

Битовые индексы

В определенных условиях битовые индексы имеют больше преимуществ, чем индексы b-дерева:

- Если таблица состоит из миллионов строк, а ключевые столбцы обладают низкой кардинальностью (то есть в пределах столбца содержится очень мало неповторяющихся значений). Например, битовые индексы обладают преимуществом перед индексами b-дерева для столбцов с информацией о половой принадлежности и семейном положении в таблице паспортных данных.
- Когда в запросах часто используется сочетание множества условий `WHERE` с оператором `OR`;
- Когда ключевые столбцы доступны только для чтения или редко обновляются.

Структура битового индекса

Битовый индекс также организован в виде b-дерева, однако в листовом узле хранятся битовые образы всех ключевых значений, а не список идентификаторов строк. Каждый бит в образе соответствует возможному идентификатору строки. Если бит установлен, значит, строка с соответствующим идентификатором содержит ключевое значение.

Как показано на схеме, листовой узел битового индекса содержит следующую информацию:

- заголовок записи, в котором содержится количество столбцов и информация о блокировании;

- ключевые значения в виде пар длина-значение для каждого ключевого столбца (в примере на слайде ключ состоит из одного столбца; в первом элементе хранится ключевое значение `Blue`).

Битовые индексы (продолжение)

Структура битового индекса (продолжение)

- Начальный `ROWID` в примере указывает на блок номер 10, строку номер 0 и файл номер 3.
- Конечный `ROWID` в примере указывает на блок номер 12, строку номер 8 и файл номер 3.
- Битовый сегмент состоит из строки битов (бит устанавливается, если соответствующая ему строка содержит ключевое значение, и не устанавливается в противном случае). Для хранения битовых сегментов сервер Oracle использует запатентованную технологию сжатия).

Начальный `ROWID` – это идентификатор первой строки, на которую указывает битовый сегмент битового образа. Соответственно, первый бит образа соответствует этой строке, второй бит – следующей строке в блоке, а конечный `ROWID` указывает на последнюю строку в таблице, входящей в битовый сегмент. В битовых индексах используются ограниченные идентификаторы строк.

Использование битовых индексов

B-дерево используется для поиска листовых узлов, содержащих битовые сегменты для заданного значения ключа. Начальный `ROWID` и битовые сегменты используются для поиска строк, содержащих ключевое значение.

Если в ключевой столбец таблицы вносятся изменения, также следует изменить битовые образы. При этом соответствующие битовые сегменты блокируются. Поскольку блокируется весь битовый сегмент, строку, входящую в битовый образ, невозможно будет обновить до завершения первой транзакции.

Параметры индекса

- Уникальный индекс: все индексированные значения должны быть уникальными.
- Индекс с реверсированным ключом: байты ключевых значений хранятся в обратном порядке.
- Составной индекс: основывается на нескольких столбцах.
- Индекс на основе функции: основывается на значении, возвращаемом функцией.
- Сжатый индекс: очищен от повторяющихся ключевых значений.
- Порядок: ключевые значения индекса могут храниться в порядке по возрастанию или убыванию.

ORACLE

8 - 34 © Oracle, 2007. Все права защищены.

Параметры индекса

Для оптимизации выборки может оказаться удобным хранить ключи в индексе в убывающем порядке. Такое решение основано на наиболее распространенной процедуре обращения к данным.

В *индексе с реверсированным ключом* байты индексированных значений хранятся в обратном порядке. Это может снизить интенсивность обращений к определенной горячей точке индекса. Если много пользователей обрабатывают данные в одинаковом порядке, то префиксные части ключевых значений (находящихся в обработке) в любой заданный момент близки по значению. Соответственно, с этой областью индекса выполняется много операций. Если используется индекс с реверсированным ключом, то эти операции распределяются по всей структуре индекса, так как индексируются ключевые значения с обратным порядком байтов.

Индекс, созданный на основе нескольких столбцов, называется *составным индексом*. Например, можно создать индекс на основе имени и фамилии.

```
CREATE INDEX name_ix ON employees
(last_name, first_name);
```

Параметры индекса (продолжение)

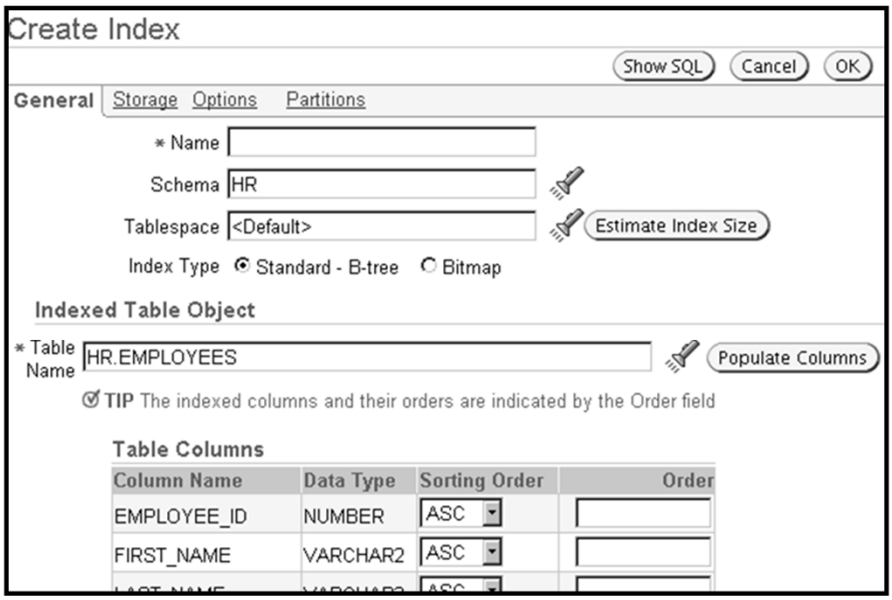
В *индексе на основе функции* индексируется возвращаемое функцией значение. Функцией может выступать стандартная функция SQL, стандартная функция PL/SQL или пользовательская функция. При этом серверу для поиска в индексированном выражении не нужно вызывать функцию для формирования каждого ключевого значения. В приведенном примере индексируются значения кубатур дерева, вычисляемых функцией на основании вида, высоты и толщины дерева (которые представлены в столбцах таблицы `TREES`):

```
CREATE INDEX tree_vol_ix ON
TREES(volume(species,height,circumference));
```

Любой запрос с выражением `volume(species,height,circumference)` в операторе `WHERE` будет обработан с использованием преимуществ данного индекса и будет выполнен намного быстрее, так как расчет кубатуры каждого из деревьев уже осуществлен. Как и обычные индексы, индексы на основе функций обслуживаются автоматически.

Сжатые индексы используются для уменьшения потребления дискового пространства во время выполнения. Так как повторяющиеся ключевые значения удалены, в заданном дисковом пространстве можно разместить большее количество элементов индекса. Это позволяет считывать с диска больше элементов индекса за то же время обращения. Для записи и чтения индекса следует выполнять соответственно сжатие и разуплотнение.

Создание индексов




```
CREATE INDEX my_index ON
employees(last_name, first_name);
```

ORACLE

8 - 36 © Oracle, 2007. Все права защищены.

Создание индексов

Чтобы открыть страницу «Indexes» (Индексы), на странице «Administration» (Администрирование) под заголовком «Schema» (Схема) щелкните ссылку «Indexes» (Индексы). Можно просмотреть атрибуты индекса или воспользоваться меню «Actions» (Действия) и просмотреть зависимости для индекса.

Индексы можно создавать явным образом, а можно – неявным, применив к таблице ограничения. Как пример неявно созданного индекса можно привести определение первичного ключа, для которого автоматически создается уникальный индекс, необходимый для принудительного определения уникальности столбца.

Представления

Схема
Ограничения
Индексы
> Представления
…

Таблица LOCATION

LOCATION_ID	STREET_ADDRESS	POSTAL_CODE	CITY	STATE_PROVINCE	CO
2200	12-98 Victoria Street	2901	Sydney	New South Wales	AU
2800	Rua Frei Caneca 1360	01307-002	Sao Paulo	Sao Paulo	BR
1000	1297 Via Cola di Rie	00989	Roma		IT
1100	93091 Calle della Testa	10934	Venice		IT

Таблица COUNTRY

CO	COUNTRY_NAME	REGION_ID
AR	Argentina	2
AU	Australia	3
BE	Belgium	1
BR	Brazil	2

Предста-
вление

LOCATION_ID	COUNTRY_NAME
2200	Australia
2800	Brazil

```
CREATE VIEW v AS SELECT location_id, country_name FROM
locations l, countries c
WHERE l.country_id = c.country_id AND c.country_id in
('AU','BR');
```

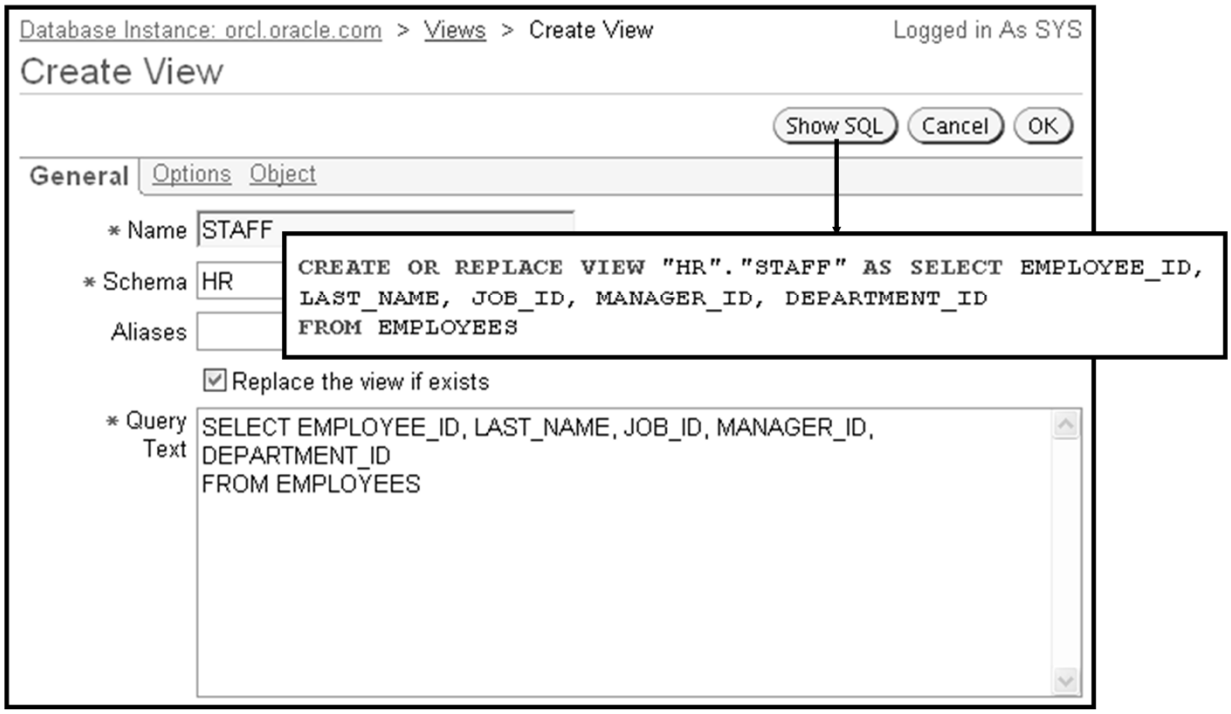
ORACLE

8 - 37 © Oracle, 2007. Все права защищены.

Представления

Представления содержат результаты запроса данных одной или более таблиц или других представлений. Представления представляют собой хранимые запросы, так как они могут скрывать очень сложные условия и объединения, а также другие сложные выражения и построения SQL. Представления не содержат данных, а извлекают эти данные из таблиц, на которых были основаны. Такие таблицы называются *базовыми таблицами* представления.

Создание представлений



ORACLE

8 - 38 © Oracle, 2007. Все права защищены.

Создание представлений

Как и таблицы, представления можно обновлять, к ним можно обращаться с помощью запросов, добавлять или удалять данные – но с некоторыми ограничениями. Все операции с представлением фактически влияют на базовые таблицы представления. Представления обеспечивают дополнительный уровень безопасности, так как ограничивается доступ к предопределенному набору строк и столбцов таблицы. Также в них скрывается сложность данных и хранятся сложные запросы.

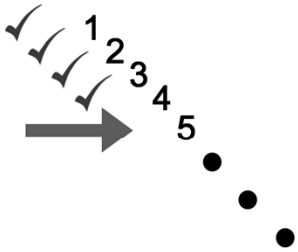
Чтобы просмотреть представления, определенные для БД, на странице «Administration» (Администрирование) под заголовком «Schema» (Схема) щелкните ссылку «Views» (Представления).

Последовательности

Схема
Ограничения
Индексы
Представления
> Последовательности
Врем. таблицы
Словарь данных

Последовательность служит для автоматического формирования целых чисел в соответствии с шаблоном.

- У последовательности есть имя, по которому выполняется обращение при запросе следующего значения.
- Последовательность не связывается с определенной таблицей или столбцом.
- Последовательность может быть возрастающей или убывающей.
- Интервал между числами может быть любым.
- При достижении предела последовательность может становиться циклической.



ORACLE

8 - 39 © Oracle, 2007. Все права защищены.

Последовательности

Чтобы получить следующее значение последовательности, необходимо использовать ссылку на ее имя, то есть связь между последовательностью и таблицей или столбцом отсутствует.

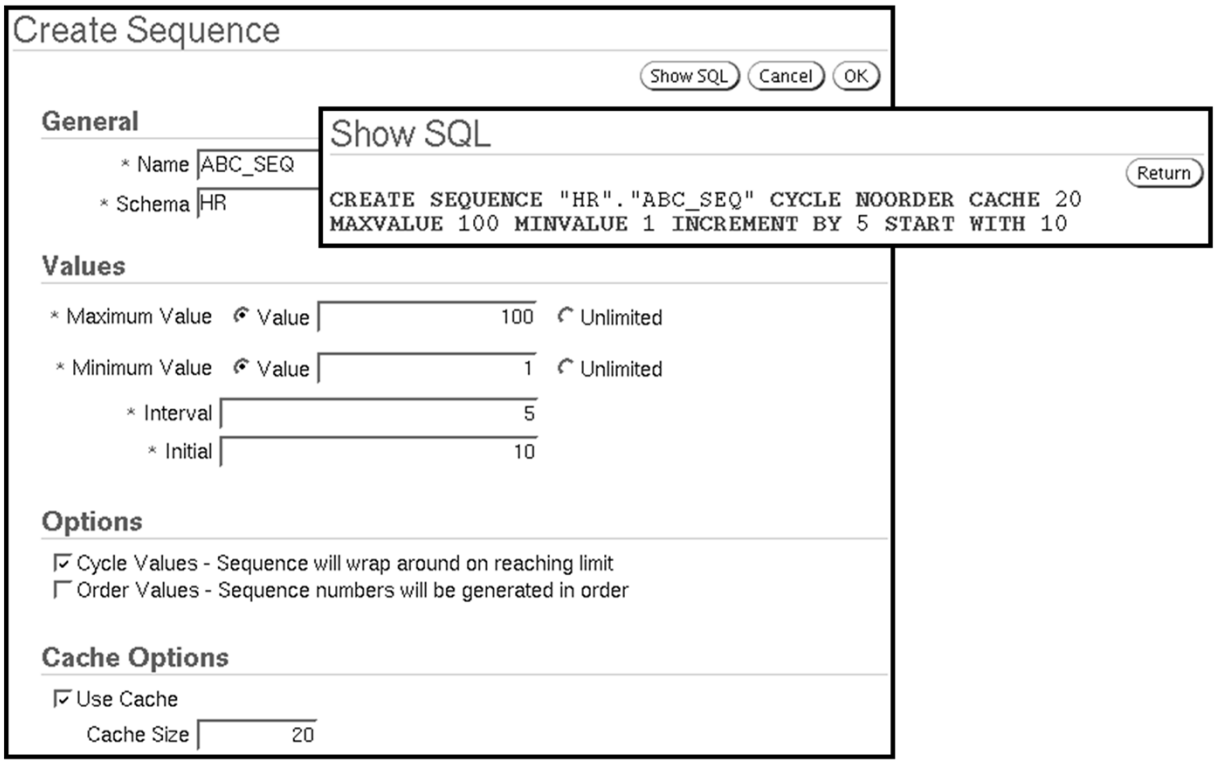
Заданное число можно получить только один раз, если только последовательность не является циклической. Иногда приложение может запросить значение, которое затем постоянно используется или хранится в базе данных. Это может привести к появлению пропусков между числами, находящимися в таблице, в которой они и сохраняются.

Кэширование чисел последовательности повышает производительность, поскольку для набора чисел предварительно выделяется память, что ускоряет доступ. При сбое экземпляра кэшированные числа последовательности не используются, что приводит к появлению пропусков.

Примечание. Если приложению требуется отсутствие пропусков, ему следует использовать нестандартный генератор чисел. Однако такой способ может катастрофически снизить производительность. Если для хранения значения используется таблица и при этом для каждого запроса приходится увеличивать значение и обновлять таблицу, этот процесс станет узким местом всей системы. Это обусловлено тем, что каждому сеансу придется ожидать обработки этим механизмом, который может одновременно обрабатывать только один запрос, чтобы обеспечить отсутствие повторяющихся значений и пропусков. Пропуски также могут появляться по истечении срока действия кэшированных значений в

разделяемом пуле. Процедура `DBMS_SHARED_POOL.KEEP` принимает параметр флага `"Q"` – это указывает, что в качестве имени хранимого объекта используется последовательность. Это предотвращает истечение срока действия последовательности.

Создание последовательности



ORACLE

8 - 40 © Oracle, 2007. Все права защищены.

Создание последовательности

Чтобы просмотреть или создать последовательность с помощью Enterprise Manager, необходимо на странице «Administration» (Администрирование) под заголовком «Schema» (Схема) щелкнуть ссылку «Sequences» (Последовательности). Ниже перечислены параметры создания последовательности.

- **Имя:** имя, которое используется в ссылках на последовательность;
- **Схема:** владелец последовательности;
- **Максимальное значение:** задает максимальное значение в последовательности. Это целое число с максимальным числом разрядов 28. Оно должно превышать минимальное и исходное значения. Значение «Unlimited» (Не ограничено) задает максимальное значение 1027 для возрастающей последовательности и –1 для убывающей. По умолчанию используется значение «Unlimited» (Не ограничено).
- **Минимальное значение:** задает минимальное значение в последовательности. Это целое число с максимальным числом разрядов 28. Оно должно быть меньше или равно исходному значению и меньше максимального значения. Значение «Unlimited» (Не ограничено) задает максимальное значение 1 для возрастающей последовательности и –1026 для убывающей. По умолчанию используется значение «Unlimited» (Не ограничено).

Создание последовательности (продолжение)

- **Интервал:** задает интервал между числами последовательности. Это целое число с любым знаком, но обязательно отличное от 0. Может содержать до 28 разрядов. По умолчанию используется значение 1.
- **Исходное значение:** задает первое число последовательности. Используется для создания возрастающей последовательности, которая начинается со значения, большего, чем минимальное. Или для создания убывающей последовательности, которая начинается со значения, меньшего, чем максимальное.
- **Циклические значения:** когда возрастающая последовательность достигает максимального значения, следующим будет снова ее минимальное значение. Когда убывающая последовательность достигает минимального значения, следующим будет снова ее максимальное значение. Если не выбрать этот параметр, попытка получить следующее значение после завершения очереди приведет к ошибке.
- **Упорядоченные значения:** числа последовательности создаются упорядоченно в порядке запросов. Это удобно при использовании чисел последовательности в качестве отметок времени. Для последовательностей, используемых для создания первичных ключей, соблюдение порядка обычно не важно. Этот параметр нужен для создания упорядоченной последовательности, только если для БД Oracle с Real Application Clusters используется параметр `CACHE`. Если кэширование не используется, последовательность упорядочивается по умолчанию.
- **Параметры кэша:** задает количество значений последовательности, для которых БД Oracle предварительно выделяет память и хранит их для ускорения доступа. Это целое число с максимальным числом разрядов 28. Минимальное значение параметра 2. Для циклических последовательностей это значение должно быть меньше количества значений в цикле. Невозможно кэшировать больше значений, чем содержится в заданном цикле чисел последовательности.

Использование последовательности

```
SQL> CREATE TABLE orders
  (id NUMBER,
   ord_date DATE,
   prod_id NUMBER,
   prod_desc VARCHAR2(30)
   );
Table created.

SQL> INSERT INTO orders VALUES ( abc_seq.NEXTVAL,
sysdate, 1245009, 'Gizmo X');

1 row created.
```

ORACLE

© Oracle, 2007. Все права защищены.

Использование последовательности

Для получения значений последовательности в SQL-операторе используются следующие псевдостолбцы:

- **CURRVAL:** возвращает текущее значение последовательности;
- **NEXTVAL:** увеличивает значение последовательности и возвращает следующее значение.

Для CURRVAL и NEXTVAL необходимо указывать имя последовательности:

последовательность.CURRVAL
последовательность.NEXTVAL

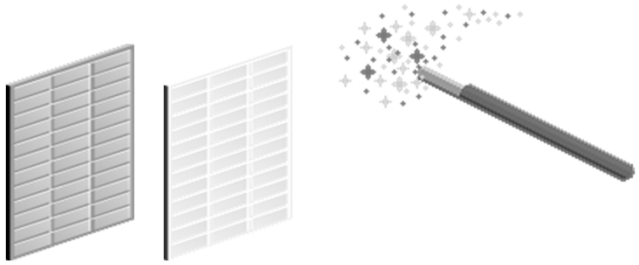
При первом использовании NEXTVAL возвращается исходное значение последовательности. При следующих обращениях к псевдостолбцу NEXTVAL значение последовательности увеличивается на заданное приращение и возвращается новое значение. Ссылка на CURRVAL всегда возвращает текущее значение последовательности, это последнее значение, которое было получено при помощи NEXTVAL.

Временные таблицы

Схема
Ограничения
Индексы
Представления
Последовательности
> Врем. таблицы
Словарь данных

Временная таблица:

- позволяет хранить данные, которые автоматически удаляются после завершения сеанса или транзакции;
- обеспечивает независимое хранение данных каждого из сеансов;
- доступна для всех сеансов, причем сохраняется независимость приватных данных каждого из них.



ORACLE

8 - 43 © Oracle, 2007. Все права защищены.

Временные таблицы

Временные таблицы обычно используют, когда для выполнения задачи необходимо сохранить частные данные, удаляемые после выполнения этой задачи, например при завершении транзакции или сеанса. Это можно сделать при помощи временных таблиц, при этом устраняется проблема скрытия частных данных от других сеансов и удаления полученных данных после завершения обработки. Единственными данными во временной таблице, видимыми для сеанса, являются данные,добавленные в этом сеансе.

Временная таблица может предназначаться для транзакции или сеанса. Во временных таблицах, предназначенных для транзакции, данные существуют, пока длится транзакция. Соответственно, во временных таблицах, предназначенных для сеанса, данные существуют, пока длится сеанс. В обоих случаях все данные, добавляемые с помощью сеанса, являются приватными данными этого сеанса. Любому сеансу доступен просмотр или изменение только собственных данных. Поэтому блокирование DML для данных из временных таблиц не применяется.

Для управления сроком существования строк используются следующие фразы:

- **`ON COMMIT DELETE ROWS`:** указывает, что время существования добавленных строк равно длительности транзакции;
- **`ON COMMIT PRESERVE ROWS`:** указывает, что время существования добавленных строк равно длительности сеанса.

Временные таблицы: анализ

- Для создания временной таблицы используется фраза **GLOBAL TEMPORARY**:

```
CREATE GLOBAL TEMPORARY TABLE employees_temp
ON COMMIT PRESERVE ROWS
AS SELECT * FROM employees;
```

- Для удаления содержимого таблицы используется команда **TRUNCATE TABLE**.
- Для временной таблицы можно создать:
  - индексы
  - представления
  - триггеры

ORACLE

8 - 44 © Oracle, 2007. Все права защищены.

Временные таблицы: анализ

Оператор `CREATE GLOBAL TEMPORARY TABLE` создает временную таблицу. Для временных таблиц можно создавать индексы, представления и триггеры. Также с помощью функций экспорта и импорта или насоса данных можно экспортировать и импортировать данные временной таблицы. Однако если использовать параметр `ROWS`, данные не будут экспортироваться.

Кроме уже описанных событий, приводящих к удалению данных, можно принудительно удалить данные с помощью команды `TRUNCATE TABLE`. При этом удаляются все добавленные пользователем данные. Эта команда более эффективна, чем команда `DELETE`.

Временную таблицу можно создать и с помощью Enterprise Manager, для этого на странице «Create Table: Table Organization» (Создание таблицы: Организация) следует выбрать параметр «Temporary» (Временная). Затем следует нажать кнопку «Continue» (Продолжить). На следующей странице можно указать, для чего предназначена временная таблица – для сеанса или транзакции. Поле «Tablespace» (Табличное пространство) недоступно, так как временная таблица всегда создается во временном табличном пространстве пользователя, а другое табличное пространство указать невозможно.

Примечание. Ключевое слово `GLOBAL` соответствует терминологии, описанной в стандарте SQL Международной организации по стандартизации (ISO).

Сводка

В этом уроке вы научились:

- определять объекты схемы и типы данных;
- создавать и изменять таблицы;
- определять ограничения;
- просматривать столбцы и содержимое таблиц;
- создавать индексы;
- создавать представления;
- создавать последовательности;
- объяснять назначение временных таблиц.

ORACLE

© Oracle, 2007. Все права защищены.

Обзор упражнения 8: администрирование объектов схемы

В данном упражнении рассматриваются следующие темы:

- создание таблиц и столбцов;
- создание ограничений:
  - `PRIMARY KEY`
  - `FOREIGN KEY`
  - `CHECK`
- создание индексов.

ORACLE

© Oracle, 2007. Все права защищены.